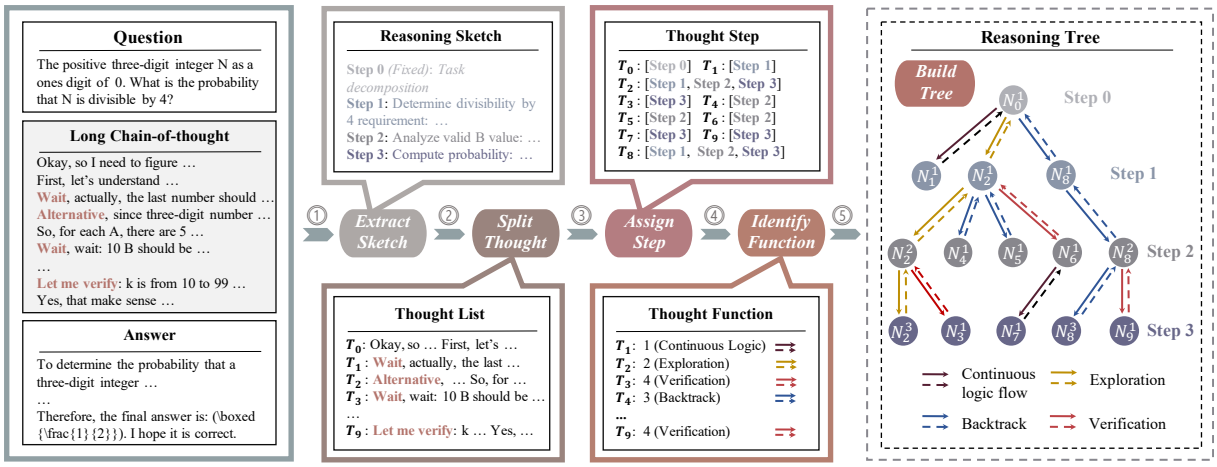

Figure 3: The workflow for LCoT2Tree. It transforms sequential long chain-of-thought into reasoning tree, which involves five steps: (1) Extract Sketch, (2) Split Thought, (3) Assign Step, (4) Identify Function, and (5) Build Tree.

3.2 LCoT2Tree Tool

We present LCoT2Tree, a novel tool that extracts structural insights from LCoTs, addressing the limitations of length-based prediction. LCoT2Tree converts the sequential chain of reasoning into a tree structure, enabling a deeper analysis of cognitive behaviors such as exploration, backtracking, and reasoning depth. These components increasingly recognized as crucial for developing reasoning LLMs (Chen et al., 2025; Gandhi et al., 2025; Ye et al., 2025). To our knowledge, our work is the first to explicitly extract this structural information and conduct a quantitative analysis of its correlation with reasoning quality.

The LCoT2Tree tool involves five automated stages that transform a LCoT into an organized tree structure using an LLM (DeepSeek-v3; Liu et al., 2024a), as shown in Figure 3:

Stage 1: Extract Sketch. Leveraging the LLM with prompting (Figure 6), we condense the LCoT into a concise *Reasoning Sketch* that outlines its main reasoning steps. The sketch serves as an abstract summary, highlighting the essential components and logical flow of the reasoning process.

Stage 2: Split Thought. We first define a "*Thought*" as a consecutive segment in the reasoning chain that involves no logical transition (*e.g.*, exploration and verification). We utilize common linguistic cues (*e.g.*, "Wait", "Alternatively", and "Let me verify") indicative of transitions between reasoning steps to segment the full reasoning chain into distinct fragments, yielding a *Thoughts List*.

Stage 3: Assign Step. Each thought in the *Thoughts List* is matched to one or more steps in the *Reasoning Sketch*, depending on its role in the overall reasoning process. This alignment is carried out using an LLM with prompting (Figure 7), generating a *Thought Step* dictionary that maps each thought to its corresponding reasoning depths.

Stage 4: Identify Function. By prompting the LLM (Figure 8), we analyze consecutive thought pairs to determine the later thought's role relative to the former, with possible roles: (1) Continuous Logic; (2) Exploration; (3) Backtracking; and (4) Verification. This assigns a *Thoughts Function* label to each thought for clearer reasoning-flow purpose understanding.

Stage 5: Build Tree. Finally, we organize the segmented thoughts into a hierarchical tree structure. Each node N_i^j in the tree corresponds to the i-th thought T_i, where j indicates how many times T_i has appeared. The placement of a node is determined by the *Thought Step*, and each edge represents a transition to a deeper level of reasoning, with the edge type defined by the *Thought Function* of its child node. When inserting a new thought T_i, we first identify the ordered list of reasoning steps it maps to, denoted as $[S_i^1, ..., S_i^n]$. Here, n indicates that the current thought encompasses n reasoning steps. Consequently, we create n nodes $N_i^1, ..., N_i^n$, where each node N_i^j represents the portion of the thought aligned with the S_i^j-th step. The insertion process follows two rules: (1) If S_i^1 is greater than the step of the latest node N_{i-1}^j in the tree, the new node N_i^1 is added as a child of N_{i-1}^j. (2) Otherwise, we backtrack to the most recent node at step $S_i^1 - 1$. Then we create a new branch from that node and link it to new node N_i^1.

Once N_i^1 is determined, the remaining nodes $N_i^2, ..., N_i^n$ are added sequentially and connected to the previous one. For example, in Figure 3, when inserting T_8 to the tree, its associated reasoning

		MATH	GPQA	LiveCodeBench	MMLU-Pro	4 Datasets	Average
DeepSeek-32B	Length-based	74.13%	67.08%	81.59%	59.95%	66.27%	69.80%
	Tree-based	80.81%	70.37%	82.21%	72.41%	71.14%	75.39%
	Gain	**+6.68%**	**+3.29%**	**+0.62%**	**+12.46%**	**+4.87%**	**+5.58%**
QwQ-32B	Length-based	75.82%	62.09%	78.30%	58.00%	66.97%	68.24%
	Tree-based	77.63%	68.55%	80.05%	72.58%	70.98%	73.96%
	Gain	**+1.81%**	**+6.46%**	**+1.75%**	**+14.58%**	**+4.01%**	**+5.72%**
DeepSeek-R1	Length-based	76.94%	69.57%	81.75%	63.00%	70.54%	72.36%
	Tree-based	80.30%	73.56%	81.80%	75.85%	73.72%	77.05%
	Gain	**+3.36%**	**+3.99%**	**+0.05%**	**+12.85%**	**+3.18%**	**+4.69%**
Seed-1.5-Thinking-pro	Length-based	67.48%	64.84%	76.39%	63.59%	64.34%	67.33%
	Tree-based	70.07%	69.81%	77.72%	70.82%	67.68%	71.22%
	Gain	**+2.59%**	**+4.97%**	**+1.33%**	**+7.23%**	**+3.34%**	**+3.89%**
Grok-3-mini-beta	Length-based	71.31%	61.48%	84.77%	55.47%	63.87%	67.38%
	Tree-based	83.18%	66.79%	86.35%	70.68%	71.26%	75.65%
	Gain	**+11.87%**	**+5.31%**	**+1.58%**	**+15.21%**	**+7.40%**	**+8.27%**

Table 1: Comparison of performance across various reasoning LLMs and datasets using the length-based method and our proposed tree-based approach for classifying response correctness based on LCoT information. Classification results are reported as the average over five runs.

steps $[S_8^1, S_8^2, S_8^3] = [1, 2, 3]$, as determined by the *Thought Step*. At that point, the latest node in tree is N_7^1, which is at step 3 (greater than 1). Therefore, we backtrack to the latest node at step 0, N_0^1, and attach N_8^1 as its child. After that, N_8^2 and N_8^3 are linked sequentially to N_8^1 and N_8^2, respectively.

In the end, we extract the tree structure, showing how thoughts are connected and branch throughout the reasoning process. This structural representation offers three key benefits: (1) highlights key cognitive patterns (*e.g.*, exploration, backtracking and verification); (2) supports more accurate assessment of reasoning quality; and (3) enables structure-aware analysis of reasoning behaviors. Implementation details, including prompts and case visualizations, are available in Appendix A.

3.3 Effectiveness of LCoT2Tree

To assess the effectiveness of the LCoT2Tree tool, we conduct a quantitative evaluation by using graph neural networks (GNNs) to predict answer correctness based on the tree structures extracted from LCoTs. This evaluation demonstrates the practical value of tree-based representations for understanding complex reasoning processes.

Experimental Setup. We use the same dataset described in Section 3.1, which contains responses from five reasoning models across four public benchmarks. The key difference is that we extract the tree structure from each LCoT response and use it as input to the GNNs. Our objective is to assess how effectively these tree structures can distinguish between correct and flawed reasoning. To this end, we utilize GATv2 (Brody et al., 2022), a GNN architecture suited for modeling hierarchical structures and their relationships. The model takes the nodes, edges, and associated features of each LCoT tree as input and learns a structural embedding that represents the overall reasoning pattern. Implementation details and graph construction are provided in Appendix B. We use classification accuracy as the evaluation metric. A high accuracy score indicates that the model successfully captures the correlation between reasoning structure and answer correctness.

Effectiveness across Tasks. Table 1 shows the classification results using tree-based input, compared to baseline methods that rely on the length-based feature. We assess how well the tree-based method generalizes across diverse types of reasoning tasks, including MATH, GPQA, LiveCodeBench, MMLU-Pro, and a combined dataset of these benchmarks. Across all tasks, the tree-based method consistently outperforms the length-based baseline. The improvement is particularly notable on MMLU-Pro, a dataset where reasoning correctness is difficult to predict from token length alone. For example, our method achieves substantial accuracy gains of +12.46% and +14.58% on DeepSeek-32B and QwQ-32B, respectively. Even on datasets like LiveCodeBench, where the length-based approach already performs strongly, the tree-based method still yields improvements, demonstrating its robustness.

Effectiveness across Models. For the generalizability of our method, the tree-based classifier con-

sistently achieves higher accuracy than the length-based baseline across all models. Average accuracy gains range from +3.89% (Seed-1.5-Thinking-pro) to +8.27% (Grok-3-mini-beta), indicating that the LCoT2Tree provides more informative and reliable structural representations of reasoning processes.

These quantitative evaluation validates the effectiveness of the LCoT2Tree tool across diverse tasks and models. By capturing deeper structural and cognitive patterns in reasoning, it enables more accurate prediction of reasoning success. Overall, LCoT2Tree shows strong potential as an automated tool for analyzing, evaluating, and improving the behavior of reasoning systems.

4 Understand Behaviors of Reasoning Large Language Models

In this section, we leverage LCoT2Tree to analyze and understand reasoning behaviors. First, we identify key thought patterns in the reasoning tree that predict errors. Then, we compare behaviors across tasks and models. Our findings show that reasoning varies by (1) output correctness, (2) task type, and (3) model variant, underscoring the importance of structural information in reasoning analysis.

Explainability Method. To interpret the model's predictions on reasoning quality and uncover the influential reasoning patterns, we adapt a graph explainability method called *GNNExplainer* (Ying et al., 2019). This method uncovers important subgraphs by maximizing the mutual information between the GNN's output and the distribution of possible subgraph structures. These extracted subgraphs also correspond to critical thought patterns within the reasoning chain. For example, in models trained to predict incorrect answers, the highlighted subgraphs often reflect flawed reasoning behaviors that lead to poor performance. Similarly, in models trained on MATH tasks, the important subgraphs typically capture common reasoning patterns observed in mathematical problem-solving.

4.1 Error Patterns in LCoT

The experiments in Section 3.3 suggest that reasoning trees of model responses exhibit separable structures for correct and incorrect outcomes. To further explore the behaviors that contribute to failures, we employ GNNExplainer to identify the most influential edges in each reasoning tree. This allows us to extract critical subgraphs from incorrect responses and summarize common patterns across

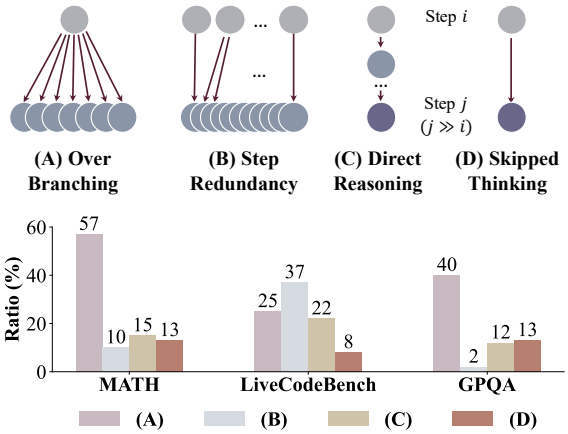

Figure 4: Visualization and frequency of four structural error patterns across three datasets. (A) *Over Branching*: abundance of explorations or verifications within a single node, (B) *Step Redundancy*: over-generation of thoughts within a single reasoning step, (C) *Direct Reasoning*: following a straight, minimal-branch path from one step to a much deeper step, and (D) *Skipped Thinking*: jumping multiple steps ahead without intermediate logical analysis. Representative example of each class is available in Figure 9.

diverse examples (See details in Appendix D.1). The identified error patterns are visualized in the top portion of Figure 4, with detailed examples shown in Figure 9. Additionally, we analyze 100 error responses from three tasks and report the frequency of each pattern in the bottom portion of Figure 4. A key observation is that excessive and insufficient branching are both strongly associated with incorrect reasoning.

4.2 Task-Specific Patterns in LCoT

In the left part of Table 2, we present the results of a task separability experiment conducted on the DeepSeek-32B model. We classify reasoning trees across task pairs (*e.g.*, MATH/GPQA, MATH/LCB, MATH/MMLU-Pro, GPQA/LCB), with additional results for QwQ-32B provided in Appendix C.1. The dataset follows the same construction as in Section 3.1, but labels tasks instead of correctness. Results show that our tree-based method effectively distinguishes task-specific reasoning patterns, achieving an average accuracy of 84.19%. Notably, in cases where length-based features fall short, such as MATH/GPQA and GPQA/LCB, tree-based representations yield substantial gains of +33.06% and +24.90%, respectively, highlighting their strength in capturing deeper reasoning patterns.

Discovering Task-Specific Reasoning Patterns. Beyond quantitative separation, we further lever-